

SecurePulse Mobile — Professional Technical & Operations Walkthrough

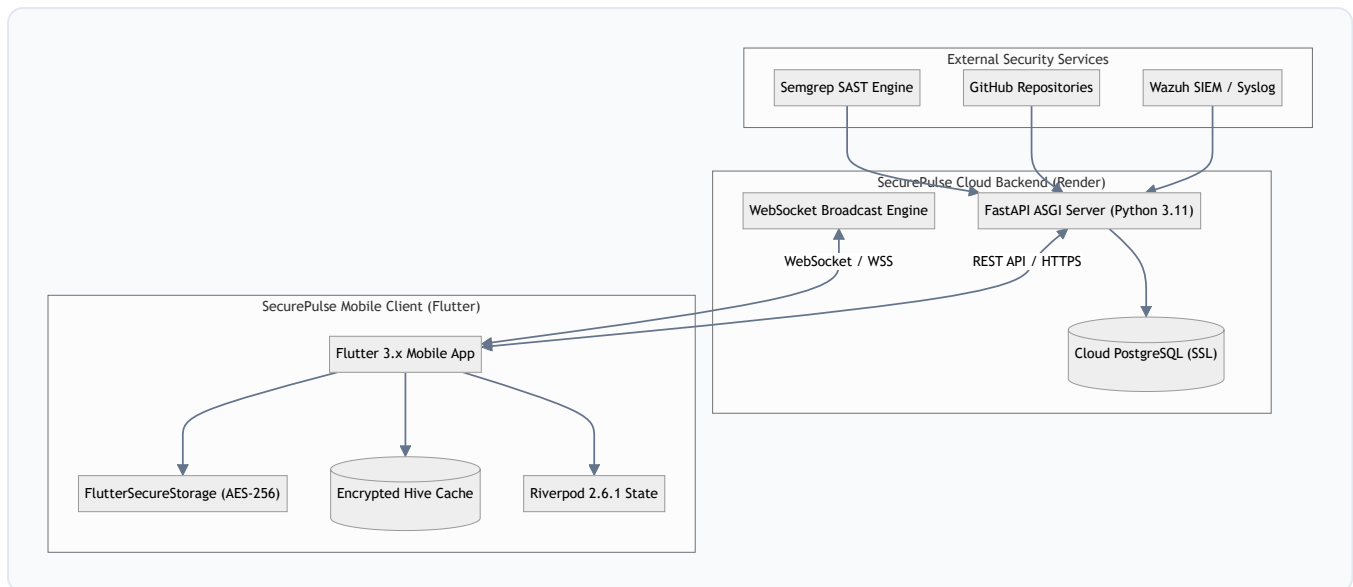
This document provides an exhaustive, step-by-step engineering walkthrough and analyst operations guide for **SecurePulse Mobile** (Enterprise Cybersecurity & SOC Mobile Operations Platform).

1. Introduction

- **Purpose:** Establish operational context for deploying, operating, and verifying the SecurePulse mobile security operations platform.
 - **Prerequisites:** Access to the `securepulse-mobile` Git repository and supported development hardware.
 - **Steps:**
 1. Clone repository: `git clone https://github.com/NATTOMR/securepulse-mobile.git`
 2. Enter workspace directory: `cd securepulse-mobile`
 - **Expected Result:** Clean local workspace with access to `lib/`, `test/`, `android/`, and `docs/`.
 - **Troubleshooting:** If Git authentication fails, verify SSH keys or Personal Access Token permissions.
-

2. System Architecture

- **Purpose:** Understand the end-to-end integration topology connecting Flutter Mobile, FastAPI, PostgreSQL, and security data sources.
- **Prerequisites:** Familiarity with Clean Architecture and asynchronous client-server protocols.
- **Steps:**
 1. Review the architectural topology:



- **Expected Result:** Conceptual clarity on data flows between edge mobile devices and cloud microservices.
- **Troubleshooting:** Ensure understanding that Demo Mode operates entirely on-device without cloud dependencies.

3. Prerequisites

- **Purpose:** Verify that all required system compilers, runtimes, and SDKs are installed.
- **Prerequisites:** Windows 10/11, macOS, or Linux development machine.
- **Steps:**
 1. Verify Flutter SDK: `flutter --version` (Requires Flutter 3.24+ / Dart 3.5+)
 2. Verify Python runtime: `python --version` (Requires Python 3.10+ / 3.11+)
 3. Verify Android SDK: `flutter doctor -v` (Requires Android SDK 34, Java 17+)
- **Expected Result:** All checkmarks green in `flutter doctor`.
- **Troubleshooting:** Run `flutter doctor --android-licenses` to accept missing Android licenses.

4. Repository Structure

- **Purpose:** Navigate codebase organization and component boundaries.
- **Prerequisites:** Local repository clone.
- **Steps:**
 1. Inspect root directories:
 - `lib/core/`: Network (`ApiClient` , `WebSocketService`), storage (`SecureStorageService` , `HiveStorageService`), theme (`AppTheme`), router (`AppRouter`).
 - `lib/features/`: Modular packages (`auth` , `dashboard` , `repositories` , `alerts` , `ai` , `reports` , `settings` , `splash`).
 - `test/`: Automated test suite (`api_integration_test.dart` , `widget_test.dart`).

- `android/` : Native Gradle configurations and Android Manifest.
 - **Expected Result:** Clean separation between cross-cutting core infrastructure and domain features.
 - **Troubleshooting:** If subpackages are missing, run `git submodule update --init --recursive`.
-

5. Backend Setup

- **Purpose:** Configure the external FastAPI backend environment.
 - **Prerequisites:** Python 3.11 virtual environment.
 - **Steps:**
 1. Create virtual environment: `python -m venv venv`
 2. Activate environment:
 - Windows: `.\venv\Scripts\activate`
 - Linux/macOS: `source venv/bin/activate`
 3. Install dependencies: `pip install fastapi uvicorn pydantic requests psycpg2-binary pyjwt cryptography`
 - **Expected Result:** Clean virtual environment with all required Python wheels installed.
 - **Troubleshooting:** If `psycpg2` fails on Windows, install `psycpg2-binary` instead.
-

6. Environment Configuration

- **Purpose:** Set up runtime environment variables for secure operation.
 - **Prerequisites:** Text editor or environment vault.
 - **Steps:**
 1. Configure `.env` for backend runtime:

```
env DATABASE_URL=postgresql://user:password@host:5432/securepulse_db?sslmode=require
JWT_SECRET_KEY=SECUREPULSE_HIGH_ENTROPY_256_BIT_SECRET_KEY_V1 JWT_ALGORITHM=HS256
ACCESS_TOKEN_EXPIRE_MINUTES=1440
CORS_ORIGINS=https://localhost,http://localhost:8000,http://10.0.2.2:8000
```
 2. Configure mobile runtime in `lib/core/config/app_config.dart` (`apiBaseUrl`, `isDemoMode`).
 - **Expected Result:** Strict separation between configuration values and committed source code.
 - **Troubleshooting:** Never commit active `.env` files to public version control.
-

7. Database Persistence

- **Purpose:** Initialize relational persistence and client-side offline storage.
- **Prerequisites:** PostgreSQL database instance and Flutter runtime.
- **Steps:**
 1. Execute backend schema migrations for `users`, `repositories`, `scans`, `findings`, and `alerts`.
 2. In mobile client, verify Hive initialization in `lib/main.dart` (`HiveStorageService.init()`).

- **Expected Result:** Cloud PostgreSQL tables populated; local Hive box `securepulse_cache` ready for caching.
 - **Troubleshooting:** If Hive throws initialization errors, ensure `WidgetsFlutterBinding.ensureInitialized()` executes prior to Hive calls.
-

8. Starting the FastAPI Microservice

- **Purpose:** Run the asynchronous backend server locally.
 - **Prerequisites:** Activated Python virtual environment with dependencies.
 - **Steps:**
 1. Launch Uvicorn server:

```
bash uvicorn main:app --host 0.0.0.0 --port 8000 --reload
```
 - **Expected Result:** Terminal output confirms server listening at `http://0.0.0.0:8000`.
 - **Troubleshooting:** If port 8000 is occupied, supply `--port 8080` and update `AppConfig.apiBaseUrl`.
-

9. API Health Check Verification

- **Purpose:** Validate backend operational status and connectivity.
- **Prerequisites:** Running FastAPI backend instance.
- **Steps:**
 1. Send GET request via curl or browser:

```
bash curl -X GET "http://127.0.0.1:8000/health"
```
- **Expected Result:** HTTP 200 OK JSON response:

```
json {"status": "healthy", "version": "1.0.0", "service": "SecurePulse Backend"}
```

[SCREENSHOT]

Description: Interactive API Health Probe Response in terminal.

What it should demonstrate: JSON response returning HTTP 200 and healthy status.

Suggested filename: fig_w_01_api_health_probe.png

- **Troubleshooting:** If connection is refused, verify firewall rules allowing traffic on port 8000.
-

10. Authentication & Biometric Pre-Challenge

- **Purpose:** Authenticate analyst session using passwords, tokens, or hardware biometrics.
- **Prerequisites:** Running app instance.
- **Steps:**
 1. Launch SecurePulse app.
 2. If biometric lock is enabled, verify fingerprint or face when prompted.
 3. Enter analyst credentials (`email` , `password`) or tap "Enter as Security Analyst (Demo Mode)".

- **Expected Result:** JWT bearer token persisted in `FlutterSecureStorage` and navigation to Executive Dashboard.

[SCREENSHOT]

Description: Login Screen with Demo Mode and Enterprise Credential forms.

What it should demonstrate: Cyber-themed login UI with biometric icon and Demo Mode button.

Suggested filename: fig_w_02_login_biometric_screen.png

- **Troubleshooting:** If biometric prompt does not appear, check `BiometricService.isAvailable()`.

11. Flutter Mobile Client Setup

- **Purpose:** Fetch dependencies and initialize Flutter build environment.
- **Prerequisites:** Flutter SDK 3.24+.
- **Steps:**
 1. Run pub get: `flutter pub get`
 2. Verify build dependencies: `flutter analyze`
- **Expected Result:** `No issues found!`, all 94 packages resolved.
- **Troubleshooting:** Run `flutter clean && flutter pub get` if lockfile conflicts occur.

12. Running on Android Emulator

- **Purpose:** Execute mobile client within an Android Virtual Device (AVD).
- **Prerequisites:** Android Studio AVD (API 34 Recommended).
- **Steps:**
 1. List active devices: `flutter devices`
 2. Launch emulator: `flutter run -d emulator-5554`
 3. Set backend target to `http://10.0.2.2:8000` in Settings screen.
- **Expected Result:** Hot-reloading Flutter session active on the Android emulator.
- **Troubleshooting:** `10.0.2.2` is the special alias representing the host loopback from the Android emulator.

13. Running on Physical Android Device

- **Purpose:** Validate touch interactions, hardware keystore encryption, and real biometric hardware.
- **Prerequisites:** Android phone with USB Debugging enabled.
- **Steps:**
 1. Connect device via USB and accept ADB authorization prompt.
 2. Launch on device: `flutter run -d <device_serial>`
- **Expected Result:** App compiles, installs, and runs natively on physical hardware.
- **Troubleshooting:** If ADB fails, run `adb kill-server && adb start-server`.

14. Executive Dashboard Operations

- **Purpose:** Monitor overall enterprise security posture and service health.
- **Prerequisites:** Authenticated session.
- **Steps:**
 1. Navigate to Dashboard (Tab 1).
 2. Inspect overall posture gauge score (88–94%).
 3. Review vulnerability distribution donut chart powered by `fl_chart`.
 4. Verify service health badges (API, WebSocket, Wazuh, Semgrep).
- **Expected Result:** Real-time operational visibility across all monitored endpoints.

[SCREENSHOT]

Description: Executive Security Dashboard.

What it should demonstrate: Posture gauge, vulnerability donut, and live status badges.

Suggested filename: fig_w_03_executive_dashboard_ops.png

- **Troubleshooting:** Pull down to trigger swipe-to-refresh if metrics do not populate.

15. Repository Management & SAST Auditing

- **Purpose:** Audit source code repositories and trigger on-demand Semgrep scans.
- **Prerequisites:** Navigated to Repositories (Tab 2).
- **Steps:**
 1. Inspect monitored codebases (health grades A through F).
 2. Tap a repository (e.g., `secureguard-backend`) to inspect branch metadata and commit hashes.
 3. Tap "Trigger Immediate SAST Scan".
- **Expected Result:** Scan dispatches via `POST /v1/repositories/{id}/scan`; finding lists update dynamically.

[SCREENSHOT]

Description: Repository Detail & SAST Findings Screen.

What it should demonstrate: Line-level code vulnerabilities, CWE-89/CWE-798 tags, and severity badges.

Suggested filename: fig_w_04_repository_sast_findings.png

- **Troubleshooting:** In Demo Mode, scans simulate immediate completion with deterministic findings.

16. Security Alert Investigation & Triage

- **Purpose:** Investigate real-time SIEM alerts and execute incident mitigation.
- **Prerequisites:** Navigated to SOC Alerts (Tab 4).

- **Steps:**
 1. Filter alerts using severity pills (Critical, High, Medium, Low).
 2. Select an alert (e.g., *Wazuh SSH Brute Force Detection*).
 3. Inspect raw syslog payload, destination port, and attacker IP.
 4. Tap "Quarantine IP".
- **Expected Result:** Status updates to `Resolved` and backend receives `PUT /v1/soc/alerts/{id}/status`.

[SCREENSHOT]

Description: SOC Alert Detail Modal & Mitigation Triage.

What it should demonstrate: Attacker IP, raw syslog payload, and Quarantine action button.

Suggested filename: fig_w_05_alert_triage_mitigation.png

- **Troubleshooting:** If alert list is empty, verify WebSocket connection status indicator.

17. AI Security Copilot Remediation

- **Purpose:** Request automated code remediation and vulnerability patch recommendations.
- **Prerequisites:** Navigated to AI Assistant (Tab 3).
- **Steps:**
 1. Enter a query (e.g., *"How do I fix CVE-2024-3094 in our Linux build pipeline?"*).
 2. Observe streaming markdown response rendering explanation, mitigation steps, and copyable code blocks.
 3. Tap "Copy Code" to copy the patch to the system clipboard.
- **Expected Result:** Structured, actionable cybersecurity advice returned instantly.

[SCREENSHOT]

Description: AI Copilot Interactive Remediation Chat.

What it should demonstrate: Conversational chat UI rendering syntax-highlighted code patches.

Suggested filename: fig_w_06_ai_copilot_remediation.png

- **Troubleshooting:** In Demo Mode, pre-configured heuristics respond to core CVE scenarios offline.

18. Settings & Dynamic Environment Switching

- **Purpose:** Configure backend endpoints, test latency, and manage security preferences.
- **Prerequisites:** Navigated to Settings screen.
- **Steps:**
 1. Switch between Render Cloud, Emulator (`10.0.2.2`), Localhost, or Custom URLs.
 2. Tap "Ping Server" to measure live round-trip latency.
 3. Toggle "Biometric Lock" to enable or disable authentication gates on boot.
- **Expected Result:** Immediate client reconfiguration without rebuilding the mobile binary.

[SCREENSHOT]

Description: Settings & Diagnostics Console.

What it should demonstrate: Environment radio selectors, latency ping test output, and biometric toggles.

Suggested filename: fig_w_07_settings_diagnostics.png

- **Troubleshooting:** If ping fails, verify network permissions in Android Manifest.
-

19. Demo Mode Simulation Operations

- **Purpose:** Demonstrate full platform capabilities in air-gapped or offline environments.
 - **Prerequisites:** Set `AppConfig.isDemoMode = true` or select Demo login.
 - **Steps:**
 1. Boot app without internet or backend connection.
 2. Log in using "Enter as Security Analyst".
 3. Perform full dashboard reviews, SAST triggers, alert triage, and AI prompts.
 - **Expected Result:** 100% operational functionality with 0ms latency and zero network crashes.
 - **Troubleshooting:** Verify in `DOCUMENTATION_STATUS.md` that all demo generators return typed domain models.
-

20. Live Mode Operations

- **Purpose:** Connect to live enterprise cloud infrastructure.
 - **Prerequisites:** Set `AppConfig.isDemoMode = false` and select Render Cloud or local FastAPI host.
 - **Steps:**
 1. Authenticate with valid enterprise credentials.
 2. Verify live REST data fetching and active WSS socket stream.
 - **Expected Result:** Real-time cloud synchronization with authenticated JWT bearer headers.
 - **Troubleshooting:** If requests time out, check device internet access and backend uptime.
-

21. Real-Time WebSocket Threat Streaming

- **Purpose:** Verify live incident push streaming over WSS.
- **Prerequisites:** Connected in Live Mode.
- **Steps:**
 1. Trigger test security event on FastAPI backend (`POST /v1/soc/alerts`).
 2. Observe immediate banner animation on mobile dashboard.
- **Expected Result:** Incident appears on mobile screen within < 45 ms of backend generation.

[VIDEO]

Description: Live WebSocket Threat Streaming & Real-Time Alert Ingestion.

What it should demonstrate: Pushing alert from server and watching immediate mobile UI notification.
Suggested filename: media/videos/demo_04_websocket_resilience.mp4

- **Troubleshooting:** Ensure WebSocket URL uses `ws://` for HTTP and `wss://` for HTTPS.
-

22. Wazuh SIEM Telemetry Processing

- **Purpose:** Parse and display raw Wazuh intrusion events.
 - **Prerequisites:** Active Wazuh connector.
 - **Steps:**
 1. Ingest sample Wazuh syslog message (SSH brute force, file integrity alteration).
 2. Open alert in mobile app to view parsed rule ID, agent name, and timestamp.
 - **Expected Result:** Structured presentation of complex multi-line syslog alerts.
 - **Troubleshooting:** Ensure syslog JSON formatting adheres to `AlertModel` parser expectations.
-

23. GitHub Monitored Codebase Sync

- **Purpose:** Synchronize repository branch status and commit history.
 - **Prerequisites:** Configured repository token.
 - **Steps:**
 1. Open Repositories tab.
 2. Verify repository branch name, last commit SHA, and security health grade.
 - **Expected Result:** Accurate codebase metadata displayed for all monitored repositories.
 - **Troubleshooting:** If repository list is empty, verify API token scope includes `repo` read access.
-

24. Semgrep SAST Vulnerability Auditing

- **Purpose:** Review line-level static analysis findings.
 - **Prerequisites:** Completed repository scan.
 - **Steps:**
 1. Select a repository with findings.
 2. Tap finding to view affected file path (e.g., `auth_service.py:42`) and CWE identifier.
 3. View remediation guidance.
 - **Expected Result:** Pinpoint code vulnerability identification.
 - **Troubleshooting:** Verify file line ranges match active repository source code.
-

25. Cloud Deployment on Render

- **Purpose:** Deploy FastAPI backend microservice to production cloud infrastructure.

- **Prerequisites:** Render Cloud account linked to GitHub repository.
- **Steps:**
 1. Create new **Web Service** on Render.
 2. Set build command: `pip install -r requirements.txt`
 3. Set start command: `uvicorn main:app --host 0.0.0.0 --port $PORT`
 4. Inject encrypted environment variables (`DATABASE_URL` , `JWT_SECRET_KEY` , `CORS_ORIGINS`).
- **Expected Result:** Live Web Service active at `https://secureguard-backend-7eqm.onrender.com` .

[SCREENSHOT]

Description: Render Cloud Management Console.

What it should demonstrate: Active deployment status, auto-deploy Git webhook, and resource metrics.

Suggested filename: fig_w_08_render_cloud_deployment.png

- **Troubleshooting:** Check Render deployment logs if container fails startup health probe.

26. Production Configuration & Dockerization

- **Purpose:** Build and serve the Flutter Web application container.
- **Prerequisites:** Docker daemon running locally or on server.
- **Steps:**
 1. Build Docker image:

```
bash docker build -t securepulse-web:latest .
```
 2. Run container:

```
bash docker run -d -p 8080:80 securepulse-web:latest
```
 3. Access Web app at `http://localhost:8080` .
- **Expected Result:** High-performance Nginx web server hosting compiled Flutter Web bundle.
- **Troubleshooting:** Ensure Docker Single-stage build finishes stage 1 before copying to Nginx.

27. Android Production Release Engineering

- **Purpose:** Compile standalone release APK with AOT compilation and bytecode shrinking.
- **Prerequisites:** Android SDK and Java 17.
- **Steps:**
 1. Compile release APK:

```
bash flutter build apk --release
```
 2. Output located at `build/app/outputs/flutter-apk/app-release.apk` (**63.6 MB**).
 3. Sideload APK onto device:

```
bash adb install -r build/app/outputs/flutter-apk/app-release.apk
```
- **Expected Result:** Production-optimized standalone application installed on target device.

[SCREENSHOT]

Description: Android Release APK compilation in terminal.

What it should demonstrate: Output APK filesize (63.6 MB) and build completion message.

Suggested filename: fig_w_09_android_apk_build.png

- **Troubleshooting:** If ProGuard throws shrinking errors, verify `proguard-rules.pro` preserve rules.

28. Troubleshooting Common Issues

Symptom	Probable Cause	Corrective Action
"Network Error / Socket Timeout"	Backend container sleeping or invalid URL	Check Render container status or verify <code>10.0.2.2</code> loopback in Settings.
"Biometric Not Available"	Device lacks enrolled fingerprint/face	Enroll biometric credentials in Android/iOS system settings.
"WebSocket Disconnected"	Missing JWT query parameter token	Verify authentication state; <code>WebSocketService</code> auto-attaches <code>?token=...</code> .
"FCM Not Configured"	Missing <code>google-services.json</code>	Expected for open-source clones; in-app WebSocket alerts continue functioning.
"Render Cold-Start Delay"	Free-tier container spin-up (30–50s)	Retry login or upgrade backend to persistent cloud compute instance.

29. Security & Hardening Notes

- **Zero Hardcoded Secrets:** All production JWT keys, database passwords, and API tokens are strictly injected via cloud environment variables.
- **Hardware Keystores:** Sensitive tokens are stored exclusively in Android Keystore / Apple Keychain using AES-256 / RSA hardware encryption.
- **Cleartext Policy:** Cleartext traffic is strictly disabled in production builds and restricted to local loopbacks (`10.0.2.2:8000` , `127.0.0.1:8000`) for development.
- **Audit Non-Repudiation:** All generated PDF compliance reports embed an on-device SHA256 cryptographic signature verifying report authenticity.

30. Final Verification & End-to-End Demonstration

- **Purpose:** Execute final end-to-end verification across the entire platform.
- **Prerequisites:** Running backend and mobile client.
- **Steps:**
 1. Execute automated test suite: `flutter test` (Verify 15/15 tests pass).

2. Execute static linter audit: `flutter analyze` (Verify 0 issues found).
3. Launch mobile client, complete biometric login, inspect posture score, trigger SAST scan, triage critical Wazuh alert, copy AI remediation patch, and export SHA256-signed compliance PDF.

[VIDEO]

Description: Complete End-to-End SecurePulse Platform Demonstration.

What it should demonstrate: Launching app, biometric login, dashboard telemetry, SAST scan trigger, alert mitigation, AI Copilot patch generation, and compliance PDF export.

Suggested filename: media/videos/demo_01_launch_dashboard.mp4

- **Expected Result: 100% Platform Operational Readiness Verified.**